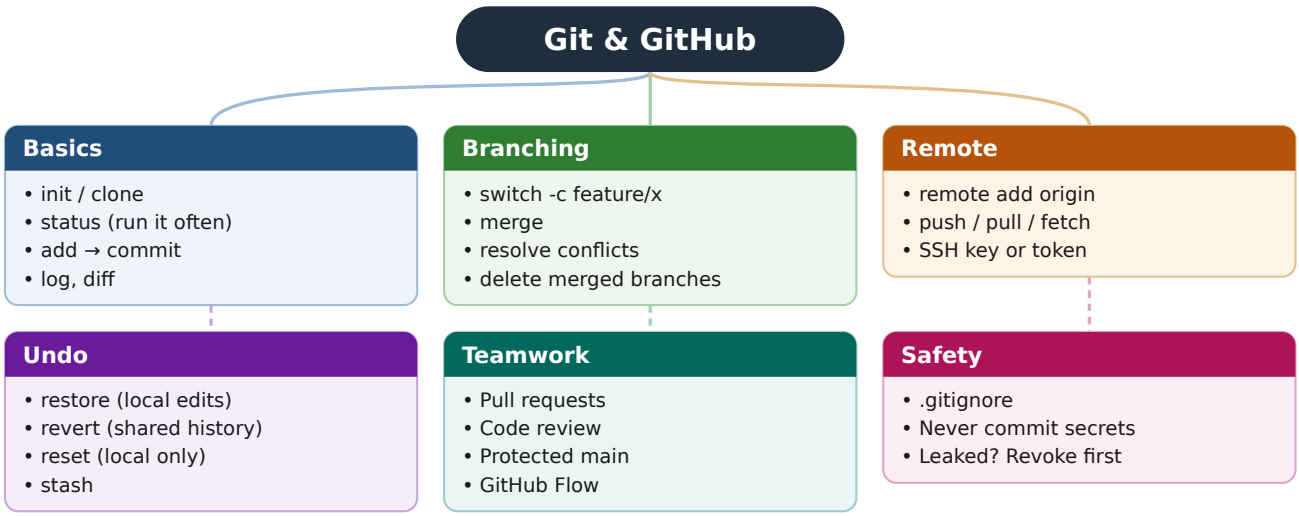
4. Git & GitHub

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 4 of 18 · Thirupathi Keloth · v2.0

Contents

4.0 Why Git matters
4.1 Version control concepts
4.2 Installing and configuring Git
4.3 How Git works internally
4.4 Everyday commands
4.5 .gitignore
4.6 Branches
4.7 Merging
4.8 Rebasing
4.9 Undoing changes and recovering work
4.10 Remotes: fetch, pull, push
4.11 Tags and releases
4.12 GitHub in depth
4.13 Branching strategies
4.14 Commits, PRs and code review practices
4.15 Repository structure choices
4.16 Hooks, pre-commit and commit signing
4.17 Secrets and Git security
4.18 Advanced and investigative commands
4.19 Troubleshooting common Git errors
4.20 Hands-on labs
4.21 Interview questions and answers
4.22 Self-check: I can...



4.0 Why Git matters

Every piece of modern infrastructure and software lives in Git: application code, Dockerfiles, Terraform, Kubernetes manifests, CI pipelines, documentation. GitOps tools deploy **from** Git. Code review, audit trails and rollbacks all depend on it. You will use Git every working day.

4.1 Version control concepts

Version control records changes to files over time so you can see history, compare versions, undo mistakes, and work in parallel with others.

Model	How	Examples
Centralized	One central server holds the history; clients check out files	SVN, Perforce
Distributed	Every clone has the **full history**; work offline; sync with remotes	**Git**, Mercurial

Git = the version control tool (created by Linus Torvalds in 2005 for the Linux kernel). **GitHub / GitLab / Bitbucket / Azure Repos** = hosting platforms for Git repositories plus collaboration features: pull requests, code review, issues, CI/CD (GitHub Actions, GitLab CI), security scanning, permissions.

4.2 Installing and configuring Git

```
git --version
git config --global user.name "Thirupathi Keloth"
git config --global user.email "you@example.com"          # use your GitHub (or noreply) email
git config --global init.defaultBranch main
git config --global core.editor "code --wait"              # or vim / nano
git config --global pull.rebase false                      # pull = fetch + merge (team convention)
git config --global core.autocrlf input                    # Linux/WSL/macOS; on Windows use: true
git config --list --show-origin                            # where each setting comes from
```

Level	File	Scope
`--system`	`/etc/gitconfig`	All users on the machine
`--global`	`~/.gitconfig`	Your user
`--local` (default)	`.git/config`	One repository (overrides global)

Useful aliases:

```
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --decorate --all"
```

4.3 How Git works internally

Git stores **snapshots**, not differences. Each commit records the complete state of the project (unchanged files are reused, so it's efficient).

Objects

Object	Contains
blob	File content
tree	A directory listing: names → blobs/trees
commit	Pointer to a root tree + parent commit(s) + author, committer, date, message
tag (annotated)	Name + message + pointer to a commit

Every object is identified by a **SHA-1 hash** of its content (e.g. `3f9a1c2…`); newer Git also supports SHA-256. Change one byte → new hash. That's why history can't be altered silently.

```
commit C3 ──parent──► commit C2 ──parent──► commit C1
   │ tree               │ tree               │ tree
   ▼                    ▼                    ▼
 snapshot             snapshot             snapshot
main ──► C3      HEAD ──► main   (HEAD = where you are now)
```

References

Ref	Meaning
Branch (`main` , `feature/login`)	A movable **pointer to a commit**; moves forward when you commit
`HEAD`	Points to the current branch (or directly to a commit = **detached HEAD**)
Tag (`v1.2.0`)	Fixed pointer to a commit (releases)
Remote-tracking branch (`origin/main`)	Your copy of where `main` was on the remote at the last fetch

The three areas and file states

```
 Working directory  ──git add──►  Staging area (index)  ──git commit──►  Repository (.git)
   (files on disk)                (the next snapshot)                     (committed history)
        ▲                                                                        │
        └──────────────────── git restore / git switch / checkout ──────────────┘

File states: untracked → (add) → staged → (commit) → unmodified → (edit) → modified → (add) → staged
```

The `.git/` directory **is** the repository: objects, refs, config, hooks. Delete it and the folder is no longer a repo.

4.4 Everyday commands

Command	Purpose
`git init`	Create a new repository in the current folder
`git clone <url> [dir]`	Copy a remote repository (full history)
`git status` / `git status -sb`	What changed, what's staged, current branch (short form)
`git add file` / `git add .` / `git add -p`	Stage a file / everything / **interactively choose hunks**
`git commit -m "Add health endpoint"`	Commit staged changes
`git commit -am "msg"`	Stage tracked modified files and commit (not new files)
`git commit --amend`	Change the last commit (message or content). Only if not pushed
`git rm file` / `git mv old new`	Delete / rename and stage it
`git restore file`	Discard working-directory changes
`git restore --staged file`	Unstage (keep changes in the working directory)

Viewing history

Command	Shows
`git log`	Full history
`git log --oneline --graph --decorate --all`	Compact branch graph
`git log -p file`	History with patches for one file
`git log --stat`	Files changed per commit
`git log --author="Thiru" --since="2 weeks ago"`	Filter by author/date
`git log -S "password"`	Commits that added/removed a string (pickaxe)
`git log main..feature`	Commits in `feature` not in `main`
`git show <commit>`	One commit's details and diff
`git show <commit>:path/file`	A file as it was in that commit

Comparing

Command	Compares
`git diff`	Working directory vs staging
`git diff --staged`	Staging vs last commit (what will be committed)
`git diff HEAD`	Working directory vs last commit
`git diff main...feature`	What the feature branch changed since it split from main
`git diff --name-only`	Just file names

4.5 .gitignore

Tells Git which **untracked** files to ignore.

```
# OS / editor
.DS_Store
.vscode/
.idea/

# Secrets and local config - never commit
.env
*.pem
*.key
credentials*

# Build outputs and dependencies
node_modules/
__pycache__/
*.pyc
dist/
build/
.venv/

# Terraform
.terraform/
*.tfstate
*.tfstate.*
*.tfvars          # if they contain environment-specific or secret values
crash.log

# Logs
*.log
!keep-this.log    # ! = exception
```

Pattern	Matches
`*.log`	Any `.log` file in any folder
`/build`	Only `build` at the repo root
`logs/`	Any directory named logs
`**/temp`	`temp` at any depth
`!file`	Do not ignore this (exception)

- Templates: github.com/github/gitignore.
- Global ignore for your machine: `git config --global core.excludesFile ~/.gitignore_global`.
- `git check-ignore -v file` explains which rule ignores a file.
- **Already committed a file?** `.gitignore` won't affect it. Stop tracking: `git rm --cached file` then commit (the file stays on disk). If it was a secret: see 4.17.

4.6 Branches

A branch is a lightweight pointer to a commit; creating one is instant. Use a branch for every change.

Command	Purpose
`git branch` / `git branch -a` / `-vv`	List local / all incl. remote / with upstream and last commit
`git switch -c feature/login`	Create and switch (older: `git checkout -b`)
`git switch main` / `git switch -`	Switch / back to previous branch
`git branch -d feature/login`	Delete a merged branch (`-D` force)

Command	Purpose
`git branch -m old new`	Rename
`git switch --detach <commit>`	Look at an old commit (detached HEAD)

Detached HEAD: HEAD points to a commit, not a branch. Commits made here are easy to lose; to keep them: `git switch -c rescue-branch`.

Branch naming conventions: `feature/JIRA-123-login-page`, `fix/null-pointer-cart`, `hotfix/1.4.1`, `chore/update-deps`, `docs/readme-install`.

4.7 Merging

Fast-forward vs three-way merge

```
FAST-FORWARD (main has no new commits):          THREE-WAY MERGE (both moved on):
main:    A──B                                     main:    A──B──C─────M   (M = merge commit,
feature:      └──D──E                                            └──D──E──┘     two parents)
after:   A──B──D──E   (main just moves forward)
```

Command	Effect
`git switch main && git merge feature/login`	Merge feature into main
`git merge --no-ff feature/login`	Always create a merge commit (keeps the branch visible in history)
`git merge --squash feature/login` + `git commit`	Combine all feature commits into one new commit
`git merge --abort`	Cancel a merge with conflicts

Resolving conflicts

A conflict happens when both sides changed the **same lines** (or one deleted a file the other changed).

```
<<<<<<< HEAD
timeout: 30
=======
timeout: 60
>>>>>>> feature/longer-timeout
```

1. `git status` lists conflicted files ("both modified").
2. Edit each file: keep the correct content, **remove all markers**.
3. `git add <file>` for each resolved file.
4. `git commit` (merge) or `git rebase --continue` (rebase).
5. Run tests before pushing.

Helpers: VS Code's merge editor, `git mergetool`, `git checkout --ours/--theirs file` (take one side entirely), `git log --merge`. **Prevent conflicts:** small PRs, merge `main` into your branch often, agree on formatting tools (auto-formatters reduce noise).

4.8 Rebasing

Rebase replays your commits on top of another base, producing a **linear history**.

```
before:  main:    A──B──C              after `git rebase main` on feature:
         feature:     └──D──E           main:    A──B──C
                                                 feature:          └──D'──E'   (new commits, new hashes)
```

Command	Purpose
`git rebase main`	Move your branch onto the latest main
`git pull --rebase`	Update your branch without merge commits
`git rebase -i HEAD~4`	**Interactive**: clean up the last 4 commits before opening a PR
`git rebase --continue / --abort / --skip`	During conflicts

Interactive rebase actions: `pick` (keep), `reword` (change message), `squash` (combine with previous, keep messages), `fixup` (combine, discard message), `drop` (remove), `edit` (stop to amend). Tip: `git commit --fixup <sha>` + `git rebase -i --autosquash`.

Golden rule: never rebase commits that others already use (shared branches like `main`). Rebasing rewrites hashes; pushing it requires a force-push and breaks collaborators' history. On your own feature branch it's fine; push with `--force-with-lease`.

	Merge	Rebase
History	True history, merge commits	Linear, cleaner
Rewrites commits	No	Yes (new hashes)
Safe on shared branches	Yes	No
Conflict resolution	Once	Possibly per replayed commit

4.9 Undoing changes and recovering work

Situation	Command
Discard uncommitted edits to a file	`git restore file`
Unstage a file	`git restore --staged file`
Fix the last commit (not pushed)	`git commit --amend`
Undo a commit that is **already pushed / shared**	`git revert <sha>` (creates a new commit that reverses it)
Undo a merge commit that is pushed	`git revert -m 1 <merge-sha>`
Move the branch back, keep changes staged	`git reset --soft HEAD~1`
Move back, keep changes unstaged (default)	`git reset HEAD~1` (`--mixed`)
Move back and **throw away** changes	`git reset --hard HEAD~1` (dangerous)
Copy one commit to the current branch	`git cherry-pick <sha>`
Find "lost" commits	`git reflog`

reset modes

```
                     branch pointer   staging area   working directory
git reset --soft X   moves to X       unchanged      unchanged      → changes appear staged
git reset --mixed X  moves to X       reset to X     unchanged      → changes appear unstaged
git reset --hard X   moves to X       reset to X     reset to X     → changes DELETED
```

reflog: the safety net

`git reflog` records every position HEAD has had (local only, kept ~90 days). After a bad reset or deleted branch:

```
git reflog                       # find the commit, e.g. a1b2c3d HEAD@{3}: commit: Add cart
git switch -c recovered a1b2c3d  # or: git reset --hard a1b2c3d
```

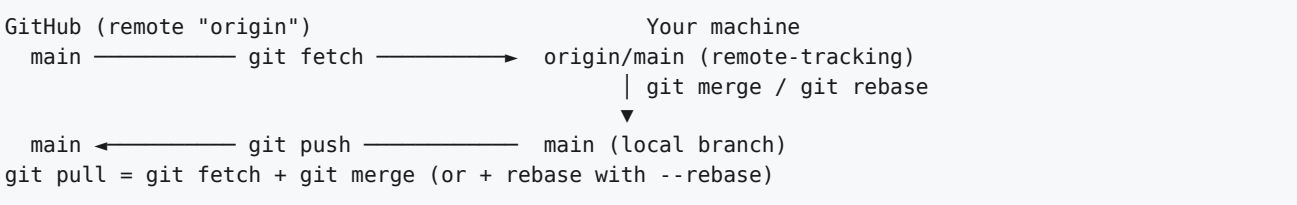
Stash: put work aside temporarily

Command	Purpose
`git stash push -m "wip login"`	Save uncommitted changes and clean the working directory
`git stash -u`	Include untracked files
`git stash list`	List stashes
`git stash pop` / `git stash apply stash@{1}`	Restore and drop / restore and keep
`git stash drop stash@{0}`	Delete a stash

Common "oops" recipes

Oops	Fix
Committed to `main` instead of a branch (not pushed)	`git switch -c feature/x` (keeps the commit) → `git switch main` → `git reset --hard origin/main`
Wrong commit message (not pushed)	`git commit --amend -m "Better message"`
Forgot a file in the last commit	`git add file && git commit --amend --no-edit`
Deleted a branch	`git reflog` → `git switch -c name <sha>`
Need a fix from another branch	`git cherry-pick <sha>`
Pushed a bad commit to `main`	`git revert <sha>` → push (never rewrite shared history)

4.10 Remotes: fetch, pull, push



Command	Purpose
`git remote -v`	List remotes

Command	Purpose
`git remote add origin git@github.com:<user>/<repo>.git`	Connect a local repo
`git remote add upstream <url>`	Original repo when you work on a fork
`git fetch` / `git fetch --all --prune`	Download changes (doesn't touch your branches) / remove deleted remote branches
`git pull`	Fetch + integrate into the current branch
`git push -u origin feature/login`	Push and set upstream tracking (first push)
`git push`	Push current branch to its upstream
`git push --force-with-lease`	Force-push **only if** nobody else pushed meanwhile (after rebasing your own branch)
`git push origin --delete feature/login`	Delete a remote branch
`git branch -u origin/main`	Set upstream for the current branch

Never use plain `--force` on shared branches. Branch protection on GitHub blocks it for `main`.

HTTPS vs SSH authentication

	HTTPS	SSH
URL	`https://github.com/user/repo.git`	`git@github.com:user/repo.git`
Auth	Personal access token (fine-grained, expiring) via credential manager; passwords not accepted	SSH key added to your GitHub account
Test	`git ls-remote <url>`	`ssh -T git@github.com`

SSH setup: `ssh-keygen -t ed25519 -C "you@example.com"` → copy `~/.ssh/id_ed25519.pub` → GitHub → Settings → SSH and GPG keys → New SSH key → `ssh -T git@github.com` → "Hi ! You've successfully authenticated".

Fork workflow (open source / restricted repos)

1. Fork the repo on GitHub → 2. clone **your fork** → 3. `git remote add upstream <original-url>` → 4. branch, commit, push to your fork → 5. open a PR to the original → 6. keep in sync: `git fetch upstream && git rebase upstream/main`.

4.11 Tags and releases

Command	Purpose
`git tag` / `git tag -l "v1.*"`	List tags
`git tag v1.2.0`	Lightweight tag (just a pointer)
`git tag -a v1.2.0 -m "Release 1.2.0"`	**Annotated** tag (author, date, message): use for releases
`git push origin v1.2.0` / `git push --tags`	Push tags (not pushed by default)
`git tag -d v1.2.0` + `git push origin --delete v1.2.0`	Delete locally and remotely
`git describe --tags`	Nearest tag + commits since (useful build versions)

Semantic versioning: `MAJOR.MINOR.PATCH`: increment MAJOR for breaking changes, MINOR for backward-compatible features, PATCH for fixes. Pre-releases: `2.0.0-rc.1`. Tags often trigger release pipelines (build image `app:1.2.0`, publish a GitHub Release with notes).

4.12 GitHub in depth

Account security

- Enable **two-factor authentication** (authenticator app or passkey/security key).
- Use SSH keys or **fine-grained personal access tokens** with expiry and minimal scopes; never classic tokens with full access.
- Review authorized OAuth apps and SSH keys periodically.
- Commit email privacy: use the `@users.noreply.github.com` address if you don't want your email public.

Repository essentials

File / feature	Purpose
`README.md`	What the project is, how to install/run/test, architecture, screenshots
`LICENSE`	Usage rights (MIT/Apache-2.0 for open source)
`.gitignore`	Ignored files
`CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`	How to contribute
`.github/CODEOWNERS`	Who must review which paths
`.github/pull_request_template.md`, issue templates	Consistent PRs and issues
`.github/workflows/*.yml`	GitHub Actions pipelines
`.github/dependabot.yml`	Automated dependency update PRs
Visibility	Public / private / internal (organizations)
Topics, description, website	Discoverability (portfolio!)

Pull requests (PRs)

```
branch → push → open PR (description, linked issue) → CI checks run → review (comments, suggestions)
      → author pushes fixes → approvals → all checks green → merge → delete branch → deploy
```

Feature	Detail
Draft PR	Work in progress; not ready for review
Reviewers / CODEOWNERS	Requested automatically for owned paths
Review types	Comment, Approve, Request changes; **suggested changes** can be applied with one click
Checks	CI status (tests, lint, security scans) shown on the PR
Linking issues	"Closes #42" in the description closes the issue on merge

Merge methods:

Method	Result	When
Create a merge commit	All commits + merge commit	Keep full history
Squash and merge	One commit on main per PR	Clean history; most common
Rebase and merge	Commits replayed linearly	Linear history with individual commits

Branch protection and rulesets (for `main`)

Require a pull request before merging · require **N approvals** · require review from **code owners** · dismiss stale approvals on new commits · require **status checks** to pass (CI) · require branches to be up to date · require **signed commits** · require linear history · block force-pushes and deletions · restrict who can push. **Rulesets** apply these across many repos/branches in an organization.

Organizations and permissions

Organizations hold repos and **teams**; permission levels per repo: Read, Triage, Write, Maintain, Admin. Use teams (not individual grants), least privilege, and SSO enforcement in companies.

Other GitHub features

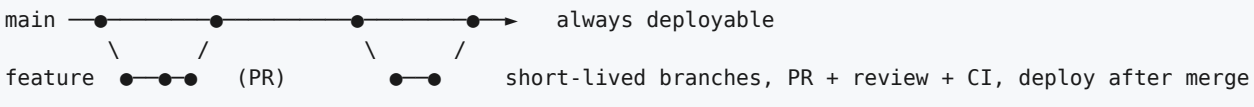
Feature	Use
Issues, labels, milestones	Track bugs and tasks
Projects	Kanban boards / planning
Actions	CI/CD (see CI/CD notes)
Packages / GHCR	Container and package registry (`ghcr.io/<owner>/<image>`)
Releases	Versioned downloads and notes from tags
Pages	Host static sites from a repo
Wiki / Discussions	Documentation / Q&A
Security: Dependabot alerts & updates, **secret scanning + push protection**, code scanning (CodeQL)	Find vulnerable dependencies, block leaked secrets, find code vulnerabilities
Codespaces	Cloud development environments

GitHub CLI (`gh`)

```
gh auth login
gh repo create my-project --private --source=. --push
gh pr create --fill --base main
gh pr checks
gh pr merge --squash --delete-branch
gh issue list --label bug
gh run list && gh run watch          # GitHub Actions runs
```

4.13 Branching strategies

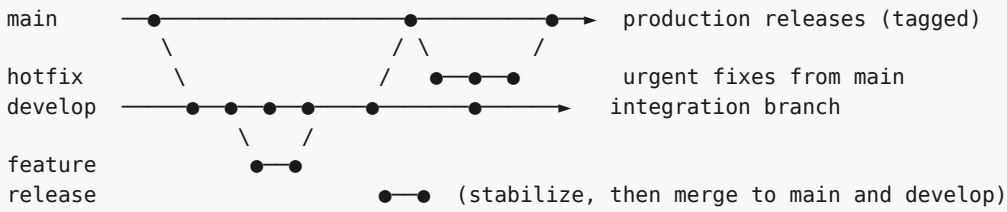
GitHub Flow (simple, most common)



Trunk-based development

Everyone integrates into `main` (the trunk) at least daily with very small changes; unfinished features are hidden behind **feature flags**; strong automated tests are required. Used by high-performing teams practicing continuous delivery.

GitFlow (heavier, release-based)



Strategy	Pros	Cons	Fits
GitHub Flow	Simple, fast, continuous deployment	Needs good CI and quick reviews	Web apps, most teams
Trunk-based	Fastest integration, fewest conflicts	Requires discipline, feature flags, excellent tests	Mature CI/CD teams
GitFlow	Structured releases, supports multiple versions	Complex, long-lived branches, more merge conflicts	Versioned products, mobile/desktop releases, regulated releases
Release branches (`release/1.4`)	Maintain older versions	Backporting work	Libraries, enterprise software

Anti-pattern: long-lived environment branches (`dev`, `qa`, `prod`) for application code: they drift apart. Promote the **same artifact** (image tag) through environments instead. (GitOps config repos may use folders per environment.)

4.14 Commits, PRs and code review practices

Good commits

- **Small and focused:** one logical change per commit.
- **Message:** imperative summary ≤ ~50 characters, blank line, then *why* (body wrapped ~72 chars).

```
Add retry with backoff to payment client

The payment provider returns 503 during deploys. Retry up to 3 times
with exponential backoff so checkout doesn't fail for users.

Closes #142
```

Conventional Commits

`type(scope): description` → `feat(api): add order history endpoint`, `fix(auth): handle expired tokens`, `docs: update install steps`, `chore(deps): bump requests to 2.32.3`, `refactor`, `test`, `ci`, `perf`, `build`. `feat!:` or a `BREAKING CHANGE:` footer marks a major change. Tools can generate changelogs and version numbers from them (semantic-release, release-please).

Pull request best practices

- Keep PRs small (a few hundred lines) and single-purpose.
- Clear description: what, why, how tested, screenshots, risk/rollback notes, linked issue.
- CI must be green before review.
- Respond to every review comment; resolve conversations.
- **Reviewers:** check correctness, tests, security (secrets, injection, permissions), readability, operability (logs, metrics, config); be specific and kind; approve when it's good enough, not perfect.

4.15 Repository structure choices

Approach	Description	Trade-offs
Polyrepo	One repo per service/component	Clear ownership, independent pipelines; harder cross-repo changes
Monorepo	Many projects in one repo	Atomic cross-project changes, shared tooling; needs path-based CI and good tooling
App repo + config repo	Code and its deployment config (Helm values, Kustomize) separated	Common with GitOps (Argo CD watches the config repo)

Submodules embed another repo at a fixed commit (`git submodule add <url> path`, clone with `--recurse-submodules`): powerful but often confusing. **Git LFS** stores large binary files outside normal history (`git lfs track "*.zip"`).

4.16 Hooks, pre-commit and commit signing

Hooks

Scripts in `.git/hooks/` that run on events (`pre-commit`, `commit-msg`, `pre-push`). They're local and not shared by default, so teams use the **pre-commit framework**:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.4
    hooks:
      - id: gitleaks
```

`pip install pre-commit && pre-commit install` → checks run before every commit; run on all files with `pre-commit run --all-files`. The same checks should also run in CI (hooks can be skipped with `--no-verify`).

Signed commits

Signing proves a commit really came from you ("Verified" badge on GitHub). Simplest with an SSH key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
# then add the same public key on GitHub as a "Signing key"
git log --show-signature -1
```

4.17 Secrets and Git security

Never commit: passwords, API keys, tokens, private keys, `.env` files, kubeconfigs, Terraform state, cloud credentials.

Prevention: `.gitignore` · pre-commit **gitleaks**/trufflehog · GitHub **secret scanning push protection** · secrets in a manager (Vault, AWS Secrets Manager, GitHub Actions secrets) · code review.

If a secret is committed (even if you "delete" it in a later commit, it stays in history):

1. **Revoke / rotate the secret immediately.** Assume it is compromised. This is the most important step.
2. Remove it from the code; load it from a secret store / environment instead.
3. Clean history if needed: `git filter-repo --path secrets.env --invert-paths` (or BFG Repo-Cleaner), then force-push and tell collaborators to re-clone. Contact GitHub support to purge cached views if the repo is public.
4. Check logs for misuse (e.g. AWS CloudTrail for leaked AWS keys).
5. Add prevention (gitignore + scanning) and write a short incident note.

4.18 Advanced and investigative commands

Command	Use
`git bisect start` → `git bisect bad` → `git bisect good v1.2.0` → test → `good` / `bad` … → `git bisect reset`	**Binary search** history to find the commit that introduced a bug (`git bisect run ./test.sh` automates it)
`git blame -L 10,30 file`	Who last changed lines 10–30 and in which commit
`git grep "TODO"`	Fast search in tracked files
`git shortlog -sn`	Commits per author
`git worktree add ../hotfix main`	Second working directory for another branch (no stash needed)
`git clean -fdn` → `-fd`	Preview / delete untracked files and directories
`git archive -o release.zip HEAD`	Export a snapshot without `.git`
`git gc` , `git count-objects -vH`	Housekeeping / repo size
`git log --follow file`	History across renames
`git range-diff`	Compare two versions of a rebased branch

4.19 Troubleshooting common Git errors

Error / situation	Cause	Fix
`! [rejected] main -> main (non-fast-forward)` / "Updates were rejected because the remote contains work"	Remote has commits you don't have	`git pull --rebase` (or `git pull`), resolve conflicts, push again
`Permission denied (publickey)`	SSH key not added to GitHub / not loaded	`ssh -T git@github.com`, add key, `ssh-add`, check `~/.ssh/config`
`remote: Support for password authentication was removed`	Using a password over HTTPS	Use a personal access token or SSH
`fatal: not a git repository`	Not inside a repo folder	`cd` into the repo or `git init`
`You are in 'detached HEAD' state`	Checked out a commit/tag	`git switch -c new-branch` to keep work, or `git switch main`
Merge conflicts	Same lines changed	Resolve markers, `git add`, commit / `rebase --continue`
`refusing to merge unrelated histories`	Two repos with separate roots (e.g. GitHub repo created with README + local init)	`git pull origin main --allow-unrelated-histories` (once), resolve, commit
`error: Your local changes would be overwritten`	Uncommitted changes conflict with switch/pull	Commit or `git stash`, then retry
File still tracked after adding to `.gitignore`	It was committed earlier	`git rm --cached file` and commit
Scripts fail with `\r` errors after editing on Windows	CRLF line endings	`core.autocrlf` settings, `.gitattributes` with `*.sh text eol=lf`, `dos2unix`
Push rejected: file > 100 MB	GitHub file size limit	Remove from history (`filter-repo`), use Git LFS or artifact storage
Pushed a secret	See 4.17	Rotate first
Branch protection blocks push to `main`	Working as designed	Open a PR

4.20 Hands-on labs

Lab 1: local workflow

```
mkdir git-lab && cd git-lab && git init
echo "# Git Lab" > README.md && git add README.md && git commit -m "Add README"
git switch -c feature/greeting
echo 'echo "Hello"' > hello.sh && git add hello.sh && git commit -m "Add greeting script"
git switch main && git merge feature/greeting       # fast-forward
git log --oneline --graph --all
```

Lab 2: create and resolve a conflict

```
git switch -c change-a && echo "timeout: 30" > app.yaml && git add . && git commit -m "Timeout 30"
git switch main && echo "timeout: 60" > app.yaml && git add . && git commit -m "Timeout 60"
git merge change-a          # CONFLICT → edit app.yaml, keep the right value, remove markers
git add app.yaml && git commit
```

Lab 3: GitHub PR flow Create a repo on GitHub → add the remote → push `main` → protect `main` (require PR + 1 approval) → push a feature branch → open a PR → review → squash-merge → pull `main` locally and delete the branch.

Lab 4: recover from mistakes — `git reset --hard HEAD~1` a commit, then recover it with `git reflog`; delete a branch and restore it.

Lab 5: rebase interactively — make 3 messy commits on a branch, run `git rebase -i HEAD~3`, squash them into one with a good message.

4.21 Interview questions and answers

#	Question	Answer
1	Git vs GitHub?	Git is the distributed version control tool; GitHub hosts Git repositories with collaboration features (PRs, Actions, issues, security)
2	What is a commit?	A snapshot of the project with metadata and parent pointer(s), identified by a hash
3	Working directory vs staging area vs repository?	Edited files / the next snapshot being prepared / committed history
4	`git fetch` vs `git pull`?	Fetch downloads without changing your branch; pull = fetch + merge (or rebase)
5	Merge vs rebase?	Merge preserves history with merge commits; rebase rewrites commits into a linear history; don't rebase shared branches
6	Fast-forward merge?	The branch pointer simply moves forward because the target has no new commits
7	How do you resolve a merge conflict?	Edit files to the correct content, remove markers, `git add`, commit/continue, run tests
8	`reset` vs `revert`?	Reset moves the branch pointer (rewrites local history); revert adds a new commit undoing changes (safe for shared branches)
9	`reset --soft` vs `--mixed` vs `--hard`?	Keep changes staged / keep unstaged / discard
10	How do you recover a deleted branch or lost commit?	`git reflog`, then create a branch at that commit
11	What is `git stash`?	Temporarily saves uncommitted changes so you can switch context
12	What is `cherry-pick`?	Applies a specific commit onto the current branch
13	What is detached HEAD?	HEAD points to a commit instead of a branch; create a branch to keep work
14	What does `.gitignore` do and why doesn't it remove a tracked file?	Ignores untracked files only; use `git rm --cached` for tracked ones

#	Question	Answer
15	What is `--force-with-lease`?	A safer force-push that fails if the remote changed since you last fetched
16	Explain GitHub Flow vs GitFlow vs trunk-based	Short branches + PRs to main / develop+release+hotfix branches / tiny frequent commits to main with feature flags
17	What is a pull request?	A request to merge a branch, with review, discussion and CI checks
18	What branch protection rules would you set on main?	Require PR + approvals + code owners, passing status checks, up-to-date branch, signed commits, no force-push/deletion
19	Squash merge vs merge commit?	One commit per PR (clean history) vs keep all commits plus a merge commit
20	Annotated vs lightweight tag?	Annotated has author/date/message (releases); lightweight is just a pointer
21	How do you find which commit introduced a bug?	`git bisect` (binary search), plus `git log -S` / `git blame`
22	What if someone committed a secret?	Rotate it immediately, remove from code, clean history (filter-repo), add scanning/push protection
23	What is a Git hook?	Script run on Git events (pre-commit, commit-msg, pre-push); shared via the pre-commit framework
24	Monorepo vs polyrepo?	One repo for many projects (atomic changes, shared tooling) vs one per service (independence, clear ownership)
25	How do you keep a fork up to date?	Add `upstream` remote, fetch it, rebase/merge `upstream/main` into your branch

4.22 Self-check: I can…

- [ ] Explain distributed version control and how Git stores data (objects, hashes, refs, HEAD).
- [ ] Configure Git (identity, editor, line endings) at the right level.
- [ ] Use the full basic cycle: status, add (incl. `-p`), commit, log, diff, show.
- [ ] Write a good `.gitignore` and untrack accidentally committed files.
- [ ] Create, switch, rename and delete branches; handle detached HEAD.
- [ ] Merge (fast-forward, three-way, squash) and resolve conflicts confidently.
- [ ] Rebase (including interactive) and explain when not to.
- [ ] Undo anything safely: restore, reset modes, revert, reflog, stash, cherry-pick.
- [ ] Work with remotes over SSH/HTTPS, fetch/pull/push, `--force-with-lease`, forks.
- [ ] Tag releases with semantic versions.
- [ ] Use GitHub PRs, reviews, merge methods, CODEOWNERS, branch protection and security features.
- [ ] Choose and explain a branching strategy for a team.
- [ ] Write good commit messages (Conventional Commits) and PR descriptions.
- [ ] Set up pre-commit hooks and signed commits.
- [ ] Respond correctly to a leaked secret.
- [ ] Use bisect, blame, worktree and fix common Git errors.

REMEMBER

- The flow: **edit → add → commit → push** (working dir → staging → repo → GitHub).
- Shared branch? Use **revert**, never reset or force-push.
- Small commits, clear messages: "Add health check endpoint".